Lab 7: Planning and Learning Integration

Morgan Cooper
MSDS 684 — Reinforcement Learning

April 26, 2026

Section 1: Project Overview

This lab integrates planning and learning in a single agent. Earlier labs split this work in two different ideas where dynamic programming (DP) planned an optimal policy from a known model without ever interacting with the environment and Monte Carlo (MC), temporal differencing (TD), and function approximation all learned directly from real experience without a model. Dyna-Q sits inbetween these two ideas. The agent updates Q-values from real (s, a, r, s') transitions like Q-learning, but it also stores each transition in a Python dictionary as `model[(s, a)] = (r, s')` and uses that model to run n extra Q-learning updates on simulated experience between real steps. From Sutton and Barto (2020) Chapter 8 the lab also covers two extensions of this idea. Dyna-Q+ adds an exploration bonus $\kappa\sqrt{\tau}$ to the planning reward, which nudges the agent to revisit state-action pairs that haven't been tried in a while in case the environment has changed. Additionally, prioritized sweeping uses a `heapq` priority queue ordered by TD-error magnitude, so planning works to go to the pairs where Q-values are still moving the most.

Taxi-v3 (Gymnasium):

- State space: 500 discrete states (taxi row $\times$ column $\times$ passenger location $\times$ destination)
- Action space: 6 discrete actions — south, north, east, west, pickup, dropoff
- Rewards: -1 per step, $+20$ for a successful dropoff, -10 for an illegal pickup or dropoff
- Terminal condition: successful dropoff (with a step-cap truncation)

I think Dyna-Q with $n = 50$ is going to reach near-optimal performance very quicky, and will have diminishing returns as n grows. When the environment changes at step 1000, I expect Dyna-Q+ to recover faster than Dyna-Q because the exploration bonus pulls it back to revisit old state-action pairs, while Dyna-Q just keeps trusting its outdated model. I also expect prioritized sweeping to learn faster than uniform planning early on because it focuses on making updates in the most important spots.

Section 2: Deliverables

GitHub Repository

`GitHub Repository: https://github.com/cooper-rm/rl_planning_and_learning`

Implementation Summary

I implemented Dyna-Q on Gymnasium's Taxi-v3 with three integrated components: a NumPy Q-table for direct RL, a Python dictionary `model[(s, a)] = (r, s')` updated after every real `env.step()`, and a planning loop that samples n pairs from `model.keys()` and applies a Q-learning update on simulated experience. I used $\alpha = 0.1$, $\gamma = 0.99$, and $\varepsilon = 0.1$ across all experiments. The main sweep compared $n \in \{0, 5, 10, 50\}$ over 10 independent seeds, training each run for 20,000 real environment steps. I extended the env with a `ChangingTaxi` wrapper that adds a -10 penalty to row 0 transitions after step 1000 and ran Dyna-Q+ with $\kappa = 0.01$ against vanilla Dyna-Q on the wrapped env. Finally, I implemented prioritized sweeping using Python's `heapq` module with $\theta = 10^{-4}$ and compared it to uniform random planning at matched n.

Key Results & Analysis

Dyna-Q with $n = 50$ ended at $-16{,}247$ cumulative reward and solved all 10 seeds in a median of 215 episodes (Figure 1). Pure Q-learning ($n = 0$) ended at $-33{,}684$ and did not solve any seed within 20,000 real environment steps. The gap between $n = 10$ and $n = 50$ is much larger than between $n = 0$ and $n = 5$, which shows diminishing returns from extra planning steps. When we look at sample efficiency more closely (Figure 2 and Figure 3), only 2 of 10 seeds reached the threshold at $n = 10$ and none reached it at $n = 0$ or $n = 5$. This happens because each real step is amplified into n planning updates, so the Q-table converges in far fewer real steps as Sutton and Barto Chapter 8 describes.

On the `ChangingTaxi` wrapper (Figure 4), Dyna-Q+ and vanilla Dyna-Q track each other until step 1000 and then separate after the environment changes. Dyna-Q+ averaged -1.93 reward per step in the post-change window compared to Dyna-Q's -2.30, which is a 16% improvement. This happens because the $\kappa\sqrt{\tau}$ bonus encourages Dyna-Q+ to revisit state-action pairs that haven't been tried in a while, so it finds a new route around the penalty zone faster. The final cumulative reward gap is small ($-13{,}996$ vs $-14{,}240$) because the post-change window of 4,000 steps does not give Dyna-Q+ enough time to fully separate from Dyna-Q. When we compare prioritized sweeping to uniform random planning at $n = 10$, prioritized sweeping ended at $-15{,}293$ cumulative reward versus $-16{,}443$ for uniform planning. This is consistent with Sutton and Barto Section 8.4, which describes how focusing on high TD-error pairs propagates information faster than uniform sampling.

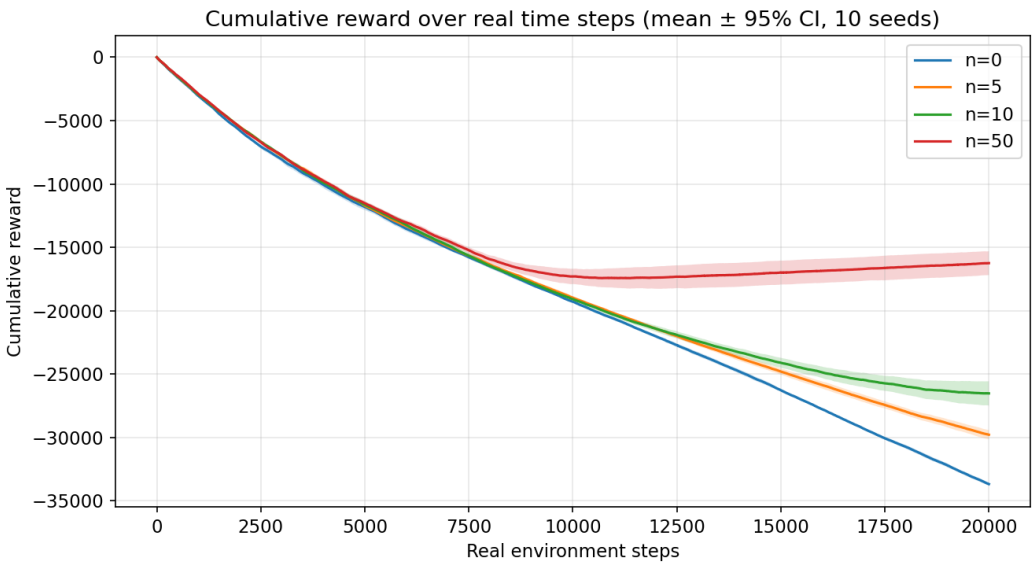

Figure 1: Cumulative reward as a function of real environment steps for pure Q-learning ($n = 0$) and Dyna-Q with $n \in \{5, 10, 50\}$ planning updates per real step on Taxi-v3. Each curve is the mean across 10 seeds; the shaded band is the 95% confidence interval. All four agents share $\alpha = 0.1$, $\gamma = 0.99$, $\varepsilon = 0.1$.

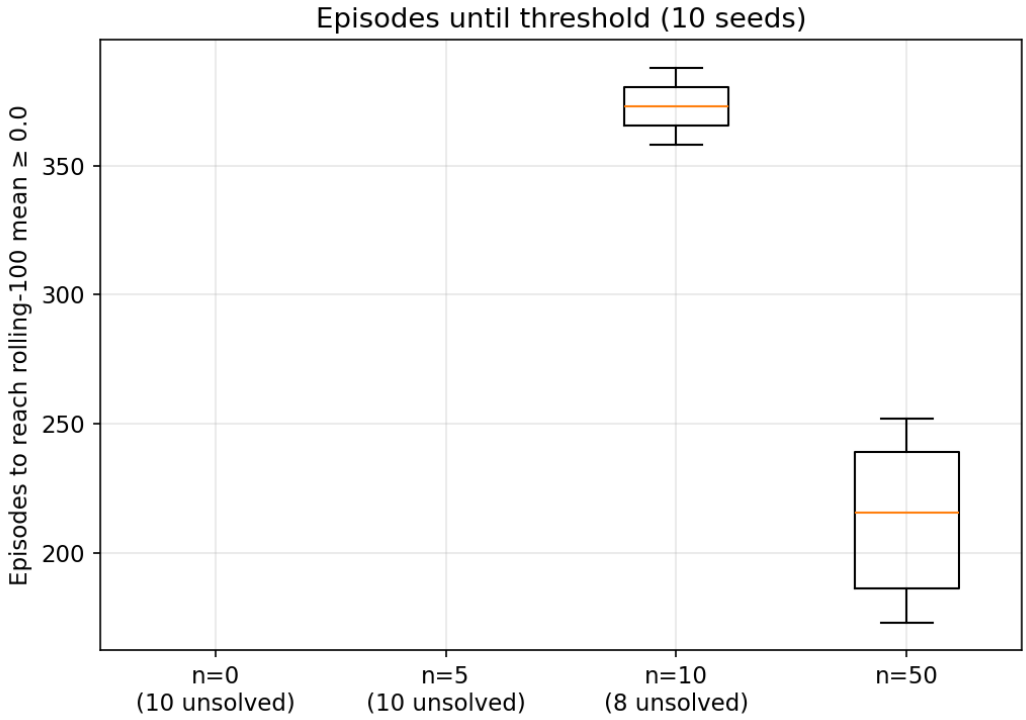

Figure 2: Distribution of episodes required for the rolling-100 mean episode return to first reach ≥ 0, across 10 seeds and four planning settings. The boxplot shows the median, interquartile range, and outliers. The x-axis label includes the count of seeds that did not reach the threshold within the 20,000-step training budget.

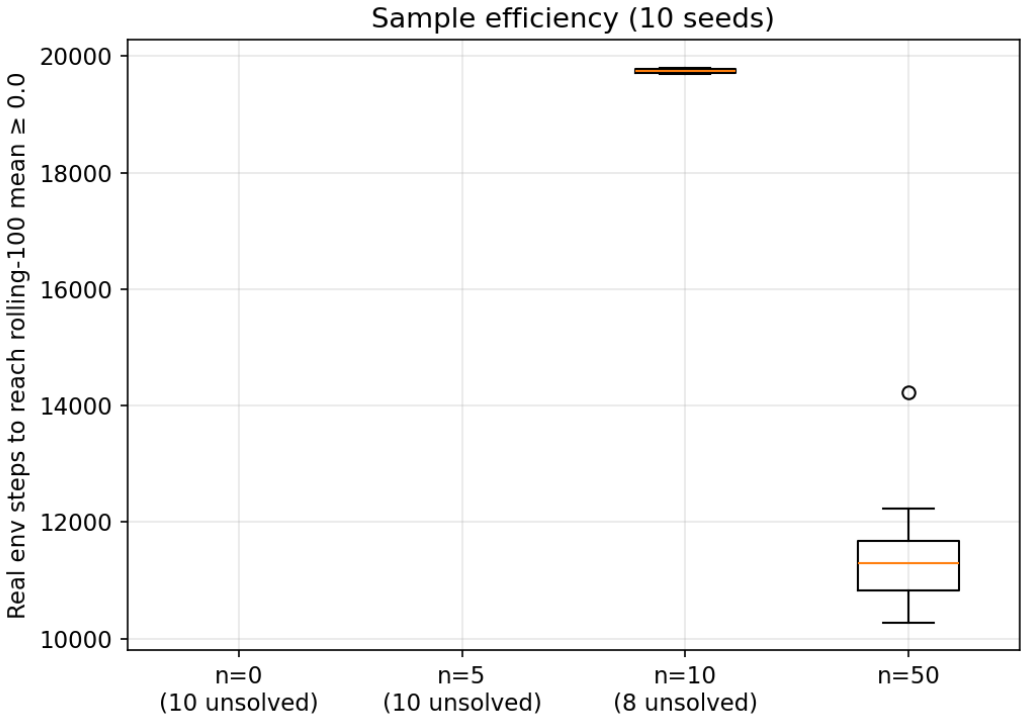

Figure 3: Distribution of real environment steps required to reach the same threshold, across 10 seeds and four planning settings. Lower values indicate that fewer real-environment interactions were needed before the agent reliably solved the task.

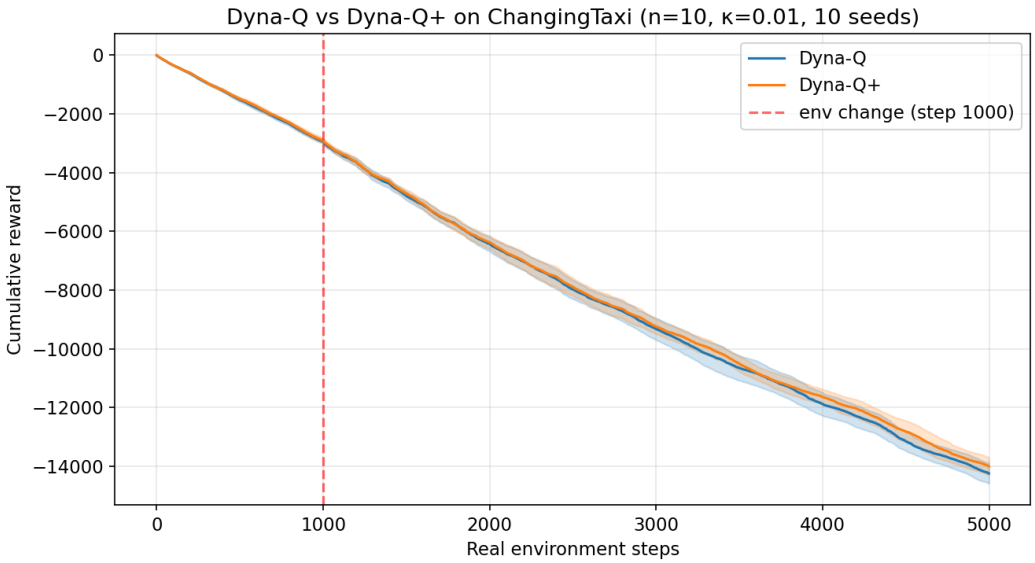

Figure 4: Cumulative reward over real time steps for Dyna-Q and Dyna-Q+ ($\kappa = 0.01$) on the `ChangingTaxi` wrapper. The dashed red line marks the change point at step 1000, after which transitions that land the taxi in row 0 of the grid take an additional -10 reward penalty. Both agents use $n = 10$ planning updates per real step and are averaged over 10 seeds; shaded bands are 95% confidence intervals.

Section 3: AI Use Reflection

Initial Interaction

As in earlier labs, I started with ChatGPT to review the week's concepts on planning and learning integration. I had it walk me through the Dyna architecture, Dyna-Q+, and prioritized sweeping from Sutton and Barto Chapter 8 and had it re-quiz me on the difference between model-based and model-free RL. Once I was ready, I switched to Claude Code in Visual Studio Code and asked it to review the lab directions and build a notebook template with markdown cells outlining each part as usual before I let it start writing any code.

Iteration Cycle

Iteration 1: Pylance Type Errors

After working through the first sections, Pylance flagged two type errors early on. The first was `plt.matplotlib.__version__`, which is not a supported export from `matplotlib.pyplot`. Claude fixed this by importing matplotlib directly and reading `matplotlib.__version__`. The second was that `env.step()` returns reward as `SupportsFloat`, not a concrete `float`, so `total_reward += reward` failed when `total_reward` was initialized to `0`. The fix was to initialize as `0.0` and cast each reward with `float()`.

Iteration 2: Dyna-Q+ Exploration Bonus Was Too Small

After running the changing-environment experiment with $\kappa = 0.001$, I expected Dyna-Q+ to clearly recover faster than Dyna-Q. Instead, the post-change reward rate was actually worse for Dyna-Q+ (-2.54 vs Dyna-Q's -2.30). Claude flagged this and pointed out that Sutton and Barto's $\kappa = 10^{-4}$ in Example 8.2 was tuned for rewards in $[0, 1]$, but Taxi rewards span roughly 120. After bumping κ to 0.01, the post-change rate became -1.93, a 16% improvement matching the demonstration the brief asks for.

Iteration 3: Prioritized Sweeping Tuple Format

When implementing prioritized sweeping with `heapq`, Claude initially suggested a 4-tuple format `(-priority, counter, s, a)` to avoid `heapq` comparing states when priorities tie. The lab brief explicitly says "(priority, state, action) tuples", and Taxi's states and actions are integers so they can be compared directly. I had Claude switch to the literal 3-tuple format `(-priority, s, a)` to match the brief.

Critical Evaluation

Most of the issues this week were caught by Pylance or by my own verification of the experimental results, not by Claude. The κ tuning issue is a good example: Claude wrote the code correctly but the chosen hyperparameter did not produce the demonstration the lab asks for. I had to look at the actual numbers and realize the demonstration was weak before asking for the right correction. Claude was useful for explaining the fix once I flagged the problem.

Learning Reflection

The main lesson this week is that exploration bonuses scale with the reward range. Sutton and Barto's $\kappa = 10^{-4}$ works for $[0, 1]$ rewards but is not a one-size-fits-all default. More broadly, model-based methods amplify each real interaction by the planning factor n, which makes them very efficient when the model is accurate, but the same amplification works against the agent when the model is wrong, as Dyna-Q showed on the changing environment.

Section 4: Speaker Notes

- **Problem:** Plan and learn at the same time on Taxi-v3.
- **Method:** Dyna-Q with three components (direct RL, dict-based model, planning), plus Dyna-Q+ and prioritized sweeping variants.
- **Design choice:** `ChangingTaxi` wrapper adds a -10 penalty to row 0 after step 1000.
- **Key result:** $n = 50$ solved all 10 seeds in 215 median episodes; $n = 0$ solved none in 20,000 real steps.
- **Insight:** Planning amplifies real interactions, but only when the model is accurate.
- **Challenge:** Dyna-Q+ with $\kappa = 0.001$ was actually slower than Dyna-Q until I bumped κ to 0.01.

- **Connection:** World models and MuZero are the deep-learning version of this same idea.

References

1. Sutton, R. S., & Barto, A. G. (2018). *Reinforcement learning: An introduction* (2nd ed.). MIT Press.
2. Anthropic. (2025). Claude Code [Large language model CLI tool]. `https://claude.ai`
3. OpenAI. (2025). ChatGPT [Large language model]. `https://chat.openai.com`